

Master-Dokumentation: 3DGS Scan-to-BIM Pipeline

KI-gestützte Vorverarbeitung, Machbarkeitsanalyse und Docker-Infrastruktur

Studiengang: Geovisualisierung (B.Eng.)
Datum: 16. Mai 2026

Inhaltsverzeichnis

1	1. Die aktuelle Pipeline-Idee (Der modifizierte 3-Schritte-Plan)	1
2	2. Machbarkeitsanalyse (Feasibility Report)	1
2.1	Phase 1: Infrastruktur (WSL2, Docker, CUDA)	1
2.2	Phase 2: Die 2D-Vorverarbeitung (SAM + DEVA)	1
2.3	Phase 3: Der fatale COLMAP-Flaschenhals	2
2.4	Phase 4: Das 3D-Meshing (SuGaR vs. Poisson)	2
3	3. Praxis-Handbuch: Docker & System-Setup	2
3.1	Administrator-Rechte	2
3.2	Abhängigkeiten & Issue #20 (DEVA)	2
3.3	Die Docker-Compose Architektur (Bauplan)	2
4	4. Repository-Analysen	3

1. Die aktuelle Pipeline-Idee (Der modifizierte 3-Schritte-Plan)

Das Ziel dieser Pipeline ist es, schwer zu erfassende, oft extrem dünne Objekte (wie Stromleitungen oder gestückelte Schläuche) präzise zu isolieren und als sauberes 3D-Gittermodell (Mesh) zu rekonstruieren. Der Workflow basiert auf einer klugen Entkopplung von 2D-Segmentierung und 3D-Training.

Die modifizierte Architektur:

1. **Das COLMAP-Fundament:** Unbearbeitete Drohnenbilder werden in COLMAP verarbeitet, um das Kamera-Tracking durch den detailreichen Hintergrund abzusichern.
2. **Intelligente 2D-Vorarbeit (Schritt 1):** Grounded-SAM (textbasiert) und DEVA (Video-tracking) weisen den Kabel-Pixeln in den Bildern ID-Masken zu.
3. **Logisches Bündeln (Schritt 2):** Ein Python-Skript färbt den Hintergrund aller Bilder rabenschwarz.
4. **Der Bait & Switch (Schritt 3):** Die Originalbilder im COLMAP-Ordner werden durch die schwarzen Bilder ausgetauscht.
5. **Fokussiertes 3D-Meshing (Schritt 4):** SuGaR trainiert 100% seiner 3D-Gaussians ausschließlich auf den Leitungen und extrahiert daraus das finale 3D-Mesh.

2. Machbarkeitsanalyse (Feasibility Report)

Phase 1: Infrastruktur (WSL2, Docker, CUDA)

Der native Windows-Betrieb (MSVC Compiler) führt bei diesen KI-Repos garantiert zu unlösbaren Kompilierungsfehlern. Windows Subsystem for Linux (WSL2) mit Ubuntu 22.04 LTS ist zwingend erforderlich. Das Problem der "Dependency Hell" (alte CUDA 11.3 Version für Vorverarbeitung vs. CUDA 11.8 für SuGaR) lässt sich **nur durch Docker** lösen. Durch die Nutzung des NVIDIA Container Toolkits laufen die Teilschritte in streng isolierten Containern. Die Masken müssen zudem nicht in einem KI-Modell gespeichert werden; sie werden effizient als PNGs in geteilten Ordnern abgelegt.

Phase 2: Die 2D-Vorverarbeitung (SAM + DEVA)

Die Out-of-the-box-Erkennung extrem dünner Leitungen scheitert oft. Grounding DINO spannt riesige Bounding Boxes um diagonale Leitungen, woraufhin SAM den Fokus verliert. DEVA verliert das Tracking bei Motion Blur. **Workarounds:** Die Bilder müssen mittels SAHI (Tiling) in Kacheln zerschnitten werden. Zur Vermeidung von Motion Blur müssen die Drohnenaufnahmen mit sehr kurzen Belichtungszeiten und hohen Framerates (60+ fps) gemacht werden.

Phase 3: Der fatale COLMAP-Flaschenhals

Ein massiver Architekturfehler im Ursprungsplan: COLMAP stürzt ab (Aperture Problem), wenn es freigestellte Bilder (Leitung auf schwarzem Hintergrund) verarbeiten soll, da SIFT keine Hintergrund-Features findet. **Die Lösung:** Der "Image Swap". COLMAP berechnet die Kamerapositionen auf den *Originalbildern*. Erst danach werden die Bilddateien auf der Festplatte gegen die schwarzen Masken ausgetauscht, bevor SuGaR gestartet wird.

Phase 4: Das 3D-Meshing (SuGaR vs. Poisson)

Die in SuGaR verwendete Poisson Surface Reconstruction baut bevorzugt geschlossene Volumen und hasst offene, hauchdünne Röhren (Gefahr des Oversmoothing). Da jedoch 100% der Gaussians (durch den schwarzen Hintergrund) auf die Leitung gezwungen werden, entsteht eine extreme Punktdichte, die Poisson stabilisiert. **Plan B:** Falls das Mesh dennoch reißt, können die Zentren der Gaussians direkt als dichte Punktwolke (.ply) an Software wie DGtal übergeben werden, um perfekte CAD-Zylinder zu fitten.

3. Praxis-Handbuch: Docker & System-Setup

Sollten Sie das Projekt auf Ihrem Windows-Laptop via WSL2 machen, oder sich einen echten Linux-Rechner (z.B. im Uni-Labor) besorgen? Ein nativer Linux-PC hat direkten GPU-Zugriff und verursacht weniger Kopfschmerzen. Müssen Sie unter Windows arbeiten, ist WSL2 der einzige Weg. Wichtig: Speichern Sie Ihre Daten niemals auf den C:/ Laufwerken, sondern zwingend innerhalb des virtuellen Linux-Dateisystems (`\\wsl$\\Ubuntu\\home\\...`).

Administrator-Rechte

Admin-Rechte unter Windows werden benötigt für:

- Die WSL2 Aktivierung (`wsl --install`).
- Die Installation von Docker Desktop für Windows (mit WSL2-Backend).

NVIDIA Treiber müssen nicht im Linux installiert werden, das übernimmt Windows automatisch.

Abhängigkeiten & Issue #20 (DEVA)

Ein berühmter C++ Compiler-Fehler (Issue #20) im DEVA-Repo unter WSL2 tritt auf, weil sich die Windows- und Linux-Umgebungsvariablen verheddern. Durch die Nutzung von Docker wird dieser Fehler obsolet, da der Container (z.B. `nvidia/cuda:11.7.1-devel-ubuntu22.04`) seinen eigenen, sauberen `nvcc` Compiler mitbringt.

Die Docker-Compose Architektur (Bauplan)

Die Trennung der Pipeline erfolgt am besten über eine `docker-compose.yml`:

```
version: '3.8'
services:
  vorverarbeitung:
    build:
      context: ./Repos_Pipeline/Tracking-Anything-with-DEVA
    volumes:
      - ./mein_drohnen_datensatz:/daten_ordner
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    command: /bin/bash

  meshing:
    build:
      context: ./Repos_Pipeline/SuGaR
    volumes:
      - ./mein_drohnen_datensatz:/daten_ordner
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    command: /bin/bash
```

4. Repository-Analysen

- **Ikeab/gaussian-grouping:** Genutzt als 2D-Vorverarbeitungs-Werkzeug (Schritt 1) zur Erstellung konsistenter Masken via SAM+DEVA, ohne das 3D-Training zu nutzen.
- **hkchengrex/Tracking-Anything-with-DEVA:** Das Herzstück des Frame-Trackings. Trennt Bildsegmentierung vom zeitlichen Tracking.

- **hkchengrex/Grounded-Segment-Anything:** Kombination aus Zero-Shot Objekterkennung (DINO) und präziser Segmentierung (SAM) per Text-Prompt (z.B. "power line").
- **Anttwo/SuGaR:** Der "FFinisher" (Schritt 4). Extrahiert ein hochpräzises 3D-Mesh aus den flach an der Oberfläche anliegenden Gaussians.
- **florinshen/FlashSplat:** Optionaler Plan-B Benchmark für extrem schnelle 2D-zu-3D Projektion.